Práctica Nº 2 - Razonamiento ecuacional e inducción estructural

Para resolver esta práctica se recomienda tener a mano las soluciones de los ejercicios de la práctica 1, así como también los apuntes de las clases teóricas y prácticas de Programación Funcional.

En las demostraciones por inducción estructural, justifique **todos** los pasos: por qué axioma, por qué lema, por qué puede aplicarse la hipótesis inductiva, etc. Es importante escribir el **esquema de inducción**, planteando claramente los casos base e inductivos, e identificando la hipótesis inductiva y la tesis inductiva.

El alcance de todos los cuantificadores que se utilicen debe estar claramente definido (si no hay paréntesis, se entiende que llegan hasta el final).

Demuestre todas las propiedades auxiliares (lemas) que utilice.

Los ejercicios marcados con el símbolo ★ constituyen un subconjunto mínimo de ejercitación. Sin embargo, aconsejamos fuertemente hacer todos los ejercicios.

Extensionalidad y lemas de generación

Ejercicio 1 ★

Sean las siguientes definiciones de funciones:

```
- intercambiar (x,y) = (y,x)
- espejar (Left x)  = Right x
  espejar (Right x) = Left x
- asociarI (x,(y,z)) = ((x,y),z)
- asociarD ((x,y),z)) = (x,(y,z))
- flip f x y = f y x
- curry f x y = f (x,y)
- uncurry f (x,y) = f x y
```

Demostrar las siguientes igualdades usando los lemas de generación cuando sea necesario:

I. ∀ `p::(a,b) . intercambiar (intercambiar p) = p`

II. ∀ `p::(a,(b,c)) . asociarD (asociarI p) = p`

III. ∀ `p::Either a b . espejar (espejar p) = p`

IV. ∀ `f::a->b->c .` ∀ `x::a .` ∀ `y::b . flip (flip f) x y = f x y`

V. ∀ `f::a->b->c .` ∀ `x::a .` ∀ `y::b . curry (uncurry f) x y = f x y`

Ejercicio 2 ★

Demostrar las siguientes igualdades utilizando el principio de extensionalidad funcional:

I. `flip . flip = id`

II. ∀ `f::(a,b)->c . uncurry (curry f) = f`

III. `flip const = const id`

IV. ∀ `f::a->b .` ∀ `g::b->c .` ∀ `h::c->d . ((h . g) . f) = (h . (g . f))`
con la definición usual de la composición: `(.) f g x = f (g x)`.

Inducción sobre listas

En esta sección usaremos las siguientes definiciones (y las de elem, foldr, foldl, map y filter vistas en clase):

```
     length :: [a] -> Int
{L0} length [] = 0
{L1} length (x:xs) = 1 + length xs

     duplicar :: [a] -> [a]
{D0} duplicar [] = []
{D1} duplicar (x:xs) = x : x : duplicar xs
```

```
      (++) :: [a] -> [a] -> [a]
{++0} [] ++ ys = ys
{++1} (x:xs) ++ ys = x : (xs ++ ys)

     append :: [a] -> [a] -> [a]
{A0} append xs ys = foldr (:) ys xs

     reverse :: [a] -> [a]
{R0} reverse = foldl (flip (:)) []
```

Ejercicio 3 ★

Demostrar las siguientes propiedades:

I. ∀ `xs::[a]` . `length (duplicar xs) = 2 * length xs`

II. ∀ `xs::[a]` . ∀ `ys::[a]` . `length (xs ++ ys) = length xs + length ys`

III. ∀ `xs::[a]` . ∀ `x::a` . `[x] ++ xs = x:xs`

IV. ∀ `xs::[a]` . `xs ++ [] = xs`

V. ∀ `xs::[a]` . ∀ `ys::[a]` . ∀ `zs::[a]` . `(xs ++ ys) ++ zs = xs ++ (ys ++ zs)`

VI. ∀ `xs::[a]` . ∀ `f::(a->b)` . `length (map f xs) = length xs`

VII. ∀ `xs::[a]` . ∀ `p::a->Bool` . ∀ `e::a` . (`elem e (filter p xs)` ⇒ `elem e xs`) (si vale `Eq a`)

Ejercicio 4 ★

Demostrar las siguientes propiedades:

I. `reverse = foldr (\x rec -> rec ++ (x:[])) []`

II. ∀ `xs::[a]` . ∀ `ys::[a]` . `reverse (xs ++ ys) = reverse ys ++ reverse xs`

III. ∀ `xs::[a]` . ∀ `x::a` . `reverse (xs ++ [x]) = x:reverse xs`

Nota: en adelante, siempre que se necesite usar `reverse`, se podrá utilizar cualquiera de las dos definiciones, según se considere conveniente.

Ejercicio 5

Demostrar las siguientes propiedades utilizando inducción estructural sobre listas, lemas de generación y el principio de extensionalidad funcional.

I. `reverse . reverse = id`

II. `append = (++)`

III. `map id = id`

IV. ∀ `f::a->b` . ∀ `g::b->c` . `map (g . f) = map g . map f`

V. ∀ `f::a->b` . ∀ `p::b->Bool` . `map f . filter (p . f) = filter p . map f`

VI. ∀ `f::a->b` . ∀ `e::a` . ∀ `xs::[a]` . (`elem e xs` ⇒ `elem (f e) (map f xs)`) (con `Eq a` y `Eq b`)

VII. ∀ `xs::[a]` . ∀ `e::a` . (`elem e xs` ⇒ `e` ≤ `maximum xs`) (si vale `Ord a`), donde:

```
     maximum :: Ord a => [a] -> a
{M0} maximum [x] = x
{M1} maximum (x:y:xs) = max x (maximum (y:xs))
```

Ayuda: usar ∀`w::Int` . ∀`x::Int` . ∀`y::Int` . ∀`z::Int` . w ≤ x ∧ y ≤ z ⇒ max w y ≤ max x z

Ejercicio 6 ★

Dadas las siguientes funciones:

```
      zip :: [a] -> [b] -> [(a,b)]
{Z0}  zip = foldr (\x rec ys ->
                    if null ys
                      then []
                      else (x, head ys) : rec (tail ys))
                  (const [])

      zip' :: [a] -> [b] -> [(a,b)]
{Z'0} zip' []     ys = []
{Z'1} zip' (x:xs) ys = if null ys then [] else (x, head ys):zip' xs (tail ys)
```

Demostrar que `zip = zip'` utilizando inducción estructural y el principio de extensionalidad.

Ejercicio 7 ★

Dadas las siguientes funciones:

```
     nub :: Eq a => [a] -> [a]
{N0} nub [] = []
{N1} nub (x:xs) = x : filter (\y -> x /= y) (nub xs)

     union :: Eq a => [a] -> [a] -> [a]
{U0} union xs ys = nub (xs++ys)

     intersect :: Eq a => [a] -> [a] -> [a]
{I0} intersect xs ys = filter (\e -> elem e ys) xs
```

Y la siguiente propiedad que vale para todos los tipos `a` y `b` pertenecientes a la clase `Eq`:
`{CONGRUENCIA ==}` $\forall$ `x::a` . $\forall$ `y::a` . $\forall$ `f::a->b` . (`a == b` $\Rightarrow$ `f a == f b`)

Indicar si las siguientes propiedades son verdaderas o falsas. Si son verdaderas, realizar una demostración. Si son falsas, presentar un contraejemplo.

I. `Eq a =>` $\forall$ `xs::[a]` . $\forall$ `e::a` . $\forall$ `p::a -> Bool` . `elem e xs && p e` = `elem e (filter p xs)`

II. `Eq a =>` $\forall$ `xs::[a]` . $\forall$ `e::a` . `elem e xs` = `elem e (nub xs)`

III. `Eq a =>` $\forall$ `xs::[a]` . $\forall$ `ys::[a]` . $\forall$ `e::a` . `elem e (union xs ys)` = `(elem e xs) || (elem e ys)`

IV. `Eq a =>` $\forall$ `xs::[a]` . $\forall$ `ys::[a]` . $\forall$ `e::a` . `elem e (intersect xs ys)` = `(elem e xs) && (elem e ys)`

V. `Eq a =>` $\forall$ `xs::[a]` . $\forall$ `ys::[a]` . `length (union xs ys)` = `length xs + length ys`

VI. `Eq a =>` $\forall$ `xs::[a]` . $\forall$ `ys::[a]` . `length (union xs ys)` $\leq$ `length xs + length ys`

Ejercicio 8

Dadas las definiciones usuales de `foldr` y `foldl`, demostrar las siguientes propiedades:

I. $\forall$ `f::a->b->b` . $\forall$ `z::b` . $\forall$ `xs, ys::[a]` . `foldr f z (xs ++ ys)` = `foldr f (foldr f z ys) xs`

II. $\forall$ `f::b->a->b` . $\forall$ `z::b` . $\forall$ `xs, ys::[a]` . `foldl f z (xs ++ ys)` = `foldl f (foldl f z xs) ys`

Otras estructuras de datos

Ejercicio 9

Demostrar que la función `potencia` definida en la práctica 1 usando `foldNat` funciona correctamente mediante inducción en el exponente.

Ejercicio 10 ★

Dadas las funciones `altura` y `cantNodos` definidas en la práctica 1 para árboles binarios, demostrar la siguiente propiedad:

$\forall$ `x::AB a . altura x` $\leq$ `cantNodos x`

Ejercicio 11

Dada la siguiente función:

```
truncar :: AB a -> Int -> AB a
{T0} truncar Nil \_ = Nil
{T1} truncar (Bin i r d) n = if n == 0 then Nil else Bin (truncar i (n-1)) r (truncar d (n-1))
```

Y los siguientes lemas:

1. $\forall$ `x::Int` . $\forall$ `y::Int` . $\forall$ `z::Int` . max (min x y) (min x z) = min x (max y z)
2. $\forall$ `x::Int` . $\forall$ `y::Int` . $\forall$ `z::Int` . z + min x y = min (z+x) (z+y)

Demostrar las siguientes propiedades:

I. $\forall$ `t::AB a` . altura t $\geq$ 0

II. $\forall$ `t::AB a` . $\forall$ `n::Int` . (`n` $\geq$ 0 $\Rightarrow$ `(altura (truncar t n) = min n (altura t))`)

Ejercicio 12

Considerar las siguientes funciones:

```
      inorder :: AB a -> [a]
  {I0} inorder = foldAB [] (\ri x rd -> ri ++ (x:rd))

      elemAB :: Eq a => a -> AB a -> Bool
  {A0} elemAB e = foldAB False (\ri x rd -> (e == x) || ri || rd)

      elem :: Eq a => a -> [a] -> Bool
  {E0} elem e = foldr (\x rec -> (e == x) || rec) False
```

Demostrar la siguiente propiedad:

`Eq a =>` $\forall$ `e::a . elemAB e = elem e . inorder`

Ejercicio 13 ★

Dados el tipo `Polinomio` y su esquema de recursión estructural `foldPoli` definidos en la práctica 1, y las siguientes funciones:

```
evaluar :: Num a => a -> Polinomio a -> a
{E} evaluar n = foldPoli n id (+) (*)

derivado :: Num a => Polinomio a -> Polinomio a
{D} derivado poli = case poli of
    X        -> Cte 1
    Cte _    -> Cte 0
    Suma p q -> Suma (derivado p) (derivado q)
    Prod p q -> Suma (Prod (derivado p) q) (Prod (derivado q) p)

sinConstantesNegativas :: Num a => Polinomio a -> Bool
{S} sinConstantesNegativas = foldPoli True (>=0) (&&) (&&)

esRaiz :: Num a => a -> Polinomio a -> Bool
{ER} esRaiz n p = evaluar n p == 0
```

Demostrar las siguientes propiedades:

I. `Num a =>` $\forall$ `p::Polinomio a` . $\forall$ `q::Polinomio a` . $\forall$ `r::a` . (`esRaiz r p` $\Rightarrow$ `esRaiz r (Prod p q)`)

II. `Num a =>` $\forall$ `p::Polinomio a` . $\forall$ `k::a` . $\forall$ `e::a` .
`evaluar e (derivado (Prod (Cte k) p))` = `evaluar e (Prod (Cte k) (derivado p))`

III. `Num a =>` $\forall$ `p::Polinomio a`. (`sinConstantesNegativas p`$\Rightarrow$`sinConstantesNegativas (derivado p)`)

La recursión utilizada en la definición de la función `derivado` ¿es estructural, primitiva o ninguna de las dos?

Ejercicio 14

Considerar las siguientes definiciones:

```
data AIH a = Hoja a | Bin (AIH a) (AIH a)

    hojas :: AIH a -> [a]
{H0} hojas (Hoja h) = [h]
{H1} hojas (Bin i d) = hojas a ++ hojas d

    espejo :: AIH a -> AIH a
{E0} espejo (Hoja h) = Hoja h
{E1} espejo (Bin i d) = Bin (espejo d) (espejo i)
```

Demostrar las siguientes propiedades:

I. $\forall$ `x::AIH a` . $\forall$ `y::AIH a` . $\forall$ `z::AIH a` . `hojas (Bin x (Bin y z)) = hojas (Bin (Bin x y) z)`

II. espejo . espejo = id

III. $\forall$ `x::AIH a` . hojas (espejo x) = reverse (hojas x)